From Anagram Reachability to Combinatorial Game Values: Termination Measures and Formalization Sketches in Lean 4
101 For DummiEz

A plain-language version of the working note

Benjamin Stanley Frohman (@Investor0x)

Prepared 19 August 2026

What This Note Is About (The Short Version)

We looked at a small computer-checked example written in the Lean 4 proof assistant. The example was about rearranging letters (anagrams) in a careful, mathematical way. We checked whether this example had already been published as a big mathematical discovery. It had not. Instead, its real value is that it teaches a clean way to prove that a computer program (or a mathematical definition) will always finish and never loop forever. That same idea turns out to be the foundation of a whole branch of mathematics called combinatorial game theory (the math behind games like Nim). This note explains the connection in everyday language.

1. The Starting Point: A Toy Anagram Example

Earlier we built a small formalization in Lean 4 around the sentence “one man’s slop is another man’s treasure.” The computer kept track of how many of each letter were left (a “count vector”). It then tried to use up those letters by subtracting the letters of real English words, one word at a time. The key safety rule was simple: every time you successfully use a word, the total number of remaining letters must go down. Because the total can only go down and cannot go below zero, the process is guaranteed to stop. That guarantee is called a termination proof.

Think of it like eating a bag of candy. Every time you eat a piece, the bag gets lighter. You cannot keep eating forever because eventually the bag is empty. The “weight of the bag” is the termination measure.

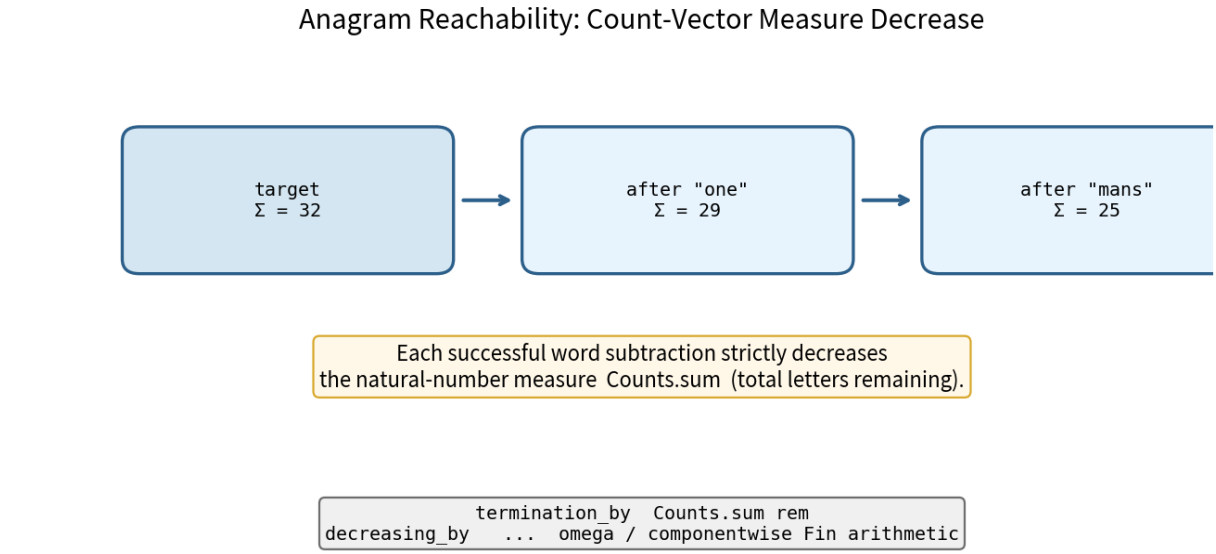


Figure 1. Each time a word is used, the total number of leftover letters gets smaller. That is the safety measure that proves the process always ends.

2. Did We Discover Something New in the Literature?

We searched published papers and existing Lean libraries. The short answer is: no, this exact anagram example was not already written up as a famous mathematical result. Related ideas exist (for example, ways of splitting numbers into smaller pieces, or covering problems in coding theory), but nothing that matches this letter-counting anagram search. So the example is original as a teaching tool, not as a breakthrough theorem that solves an open problem.

Why that matters: It keeps the claim honest. The value is in the clear demonstration of a technique, not in pretending we solved a long-standing puzzle.

3. What “Termination Measures” Mean in Lean (Plain English)

Lean is a computer program that checks mathematical proofs and also runs functional programs. When you write a recursive definition (a definition that refers to itself), Lean demands proof that the recursion cannot go on forever. You give it two things:

- A **measure** — a number (or something that can be ordered) that gets smaller each time the definition calls itself.
- A short argument showing that the measure really does get smaller on every recursive call.

In the anagram example the measure was simply “how many letters are still left.” Because every useful word removes at least one letter, the number goes down and the process must finish. Lean accepts the definition only after it sees that proof.

4. The Surprising Link to Games

Combinatorial game theory is the mathematics of two-player games with no chance and no hidden information (examples: Nim, many puzzles, some board games). The basic idea is that every position in a game can be described by the positions you can move to next (the “options”).

To define the value of a game, or to prove things about it, mathematicians use a principle called Conway induction. It works because every move takes you to a “simpler” or “earlier” position. You can rank positions by how early they appear in the universe of all games (sometimes called the birthday of the position). Every option has a strictly smaller birthday than the position you started from. That ranking is a well-founded order — exactly the same kind of “always gets smaller” idea used in the anagram example.

So the candy-bag measure (letter count) and the game-birthday measure are two faces of the same logical pattern: if something keeps getting smaller in a well-ordered way, the process cannot continue forever and induction is safe.

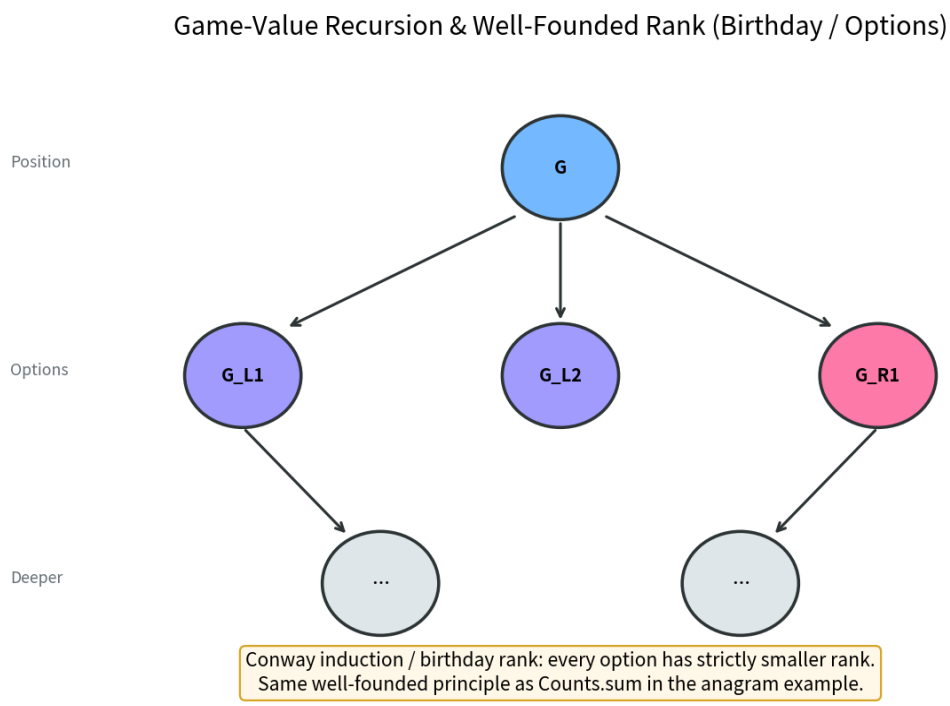


Figure 2. From a game position you can move to simpler positions. Those simpler positions have smaller “birthdays,” so the ranking never goes in a circle.

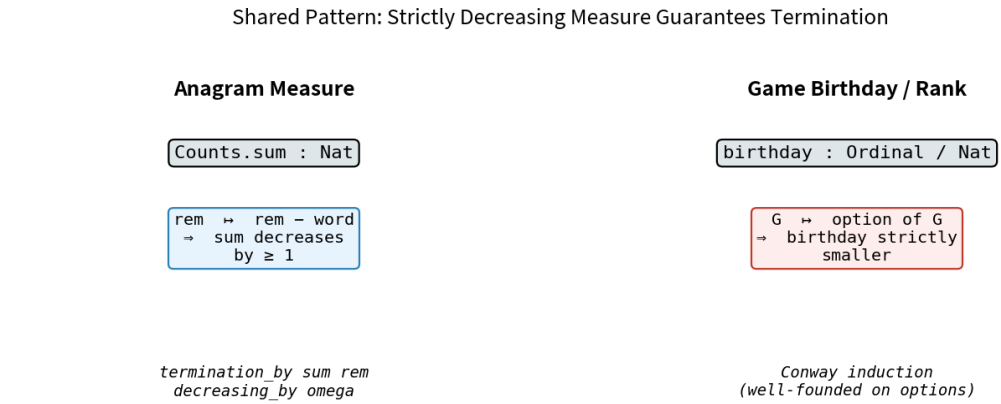


Figure 3. Side-by-side view of the two measures. One uses remaining letters; the other uses the age of a game position. Both guarantee that recursion is safe.

5. A Tiny Sketch of the Game Theory Side

In the impartial version of the theory (both players have the same moves available), every position can be given a single number called its Grundy number (or nimber). You compute it by looking at the Grundy numbers of the positions you can move to and taking the smallest missing non-negative integer (the “mex”). Because you only look at simpler positions, the calculation is well-defined and always finishes. The famous Sprague–Grundy theorem then says that any impartial game is equivalent to a single heap of Nim whose size is exactly that Grundy number.

The computer-checked libraries that formalize this theory in Lean rely on the same style of termination argument we practised with the anagram counts. Once you have seen the pattern on a small letter-counting example, the

game-theory version feels much less mysterious.

6. Why This Matters (The Take-Away)

The anagram formalization did not solve a famous open problem. Its significance is practical and educational:

- It gives a completely transparent example of a termination proof that Lean accepts.
- It shows that the same logical idea appears, at a larger scale, in the foundations of combinatorial game theory.
- For anyone learning to formalize mathematics (or preparing larger formal developments), small zero-error examples of well-founded recursion are excellent practice.

In short: the candy bag empties, the game positions get younger, and both times the process is guaranteed to stop. That single idea is the bridge this note tries to make obvious.

This is a plain-language companion to the more technical working note. All final authorship of any formal developments remains with Benjamin Stanley Frohman.